# CyberSecIntel
# P2 Final Delivery
# Trusted Zone, Exploitation Zone, Consumption, and Governance

Dídac Cayuela and Sindri Masson
Big Data Management – Universitat Politecnica de Catalunya

June 2026

## 1 Instructions

CyberSecIntel is a cybersecurity data platform for small SOC workflows. P1 implemented ingestion and a governed Landing/Silver lakehouse: public vulnerability feeds, IOC feeds, CTU-13 PCAP artifacts, Suricata replay outputs, and Kafka topics are stored in MinIO and Delta Lake. P2 completes the architecture required by the statement: Trusted Zone, Exploitation Zone, downstream consumption, and governance.

**Repository access.** The project repository is:

https://github.com/didicayu/BDM_P1_Cayuela_Masson.git

The final P2 implementation is on branch `main`. A supervisor can clone and run it locally:

```
git clone https://github.com/didicayu/BDM_P1_Cayuela_Masson.git
cd BDM_P1_Cayuela_Masson
cp .env.example .env
```

**Run order.**

1. Copy configuration: `cp .env.example .env`; optional keys are `NVD_API_KEY`, `ABUSE_CH_API_KEY`, and `SHODAN_API_KEY`.
2. Start infrastructure: `docker compose build airflow-webserver`; `docker compose up -d minio zookeeper kafka postgres grafana`; `docker compose run --rm airflow-init`; `docker compose up -d airflow-webserver airflow-scheduler`.
3. Trigger `cybersecintel_end_to_end`. This parent DAG waits for API ingestion, warm aggregate materialization, Trusted cleaning, Exploitation, and Consumption in that order. The Consumption DAG also publishes the Grafana Postgres serving tables.
4. Optionally run `cybersecintel_dataset_artifact_ingestion` and `cybersecintel_pcap_replay`; replay remains intentionally gated by `PCAP_REPLAY_ENABLED`.

The P2 stages can also be executed locally after P1 Delta tables exist. The helper translates Docker-internal service names to published localhost ports and uses Delta as the deterministic warm aggregate source:

```
scripts/run_p2_local.sh
scripts/run_p2_local.sh --spark
```

The outputs are inspected in MinIO buckets `s3://trusted/` and `s3://exploitation/`, local files under `consumption/outputs/`, and Grafana at `http://localhost:3000` after starting `postgres` and `grafana`.

**Submission evidence.** The final archive contains the source code, report source, generated PDF, tests, Docker/Airflow definitions, the trained Isolation Forest artifacts, Grafana provisioning, and generated consumption files. The verification run uses the same Docker Compose stack as the coursework demo, so the evidence is not limited to isolated unit tests.

## 2 Architecture Design

The final architecture follows Option 1 from the P2 statement: extend the P1 lakehouse instead of adding several specialized databases. This is appropriate because the project has heterogeneous cybersecurity sources but moderate volume, and the main need is reproducible movement through zones, not ultra-low-latency OLAP serving. MinIO remains the object store and Delta Lake remains the table format for all checkpoints.

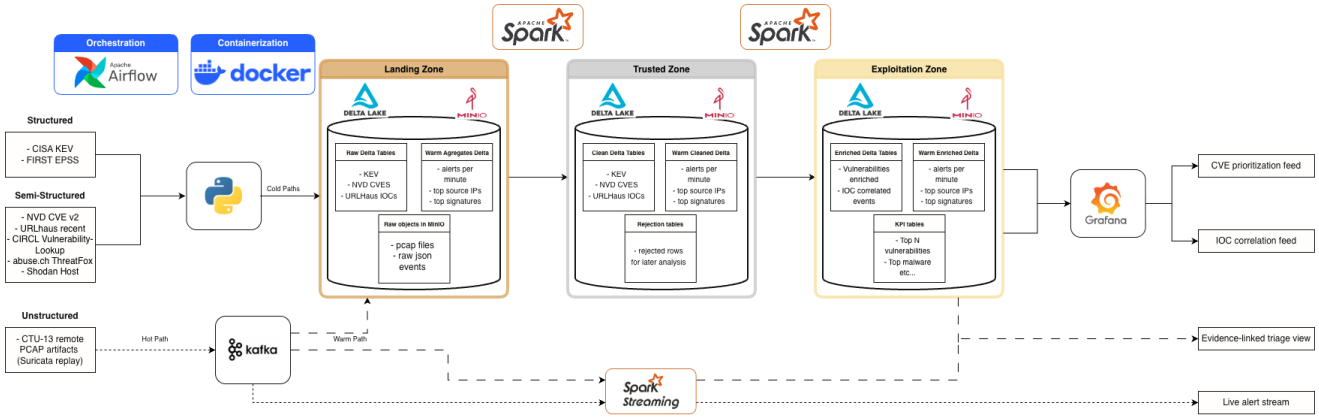


Figure 1: Final P2 architecture. P1 feeds Landing/Silver; P2 adds Trusted, Exploitation, Consumption, and Governance checkpoints.

| Stage | Implemented role |
|---|---|
| Landing/Silver | P1 raw MinIO prefixes and Delta tables: KEV, NVD, EPSS, URLhaus, CIRCL, ThreatFox, optional Shodan, PCAP catalogs, replay runs, Suricata events, and IDS alerts. |
| Trusted | `trusted/` package reads `s3://deltalake/`, performs information-preserving cleaning, writes cleaned tables and rejected-row tables to `s3://trusted/`. |
| Exploitation | `exploitation/` package writes analyst-ready Delta assets to `s3://exploitation/`: CVE enrichment, network events, IOC correlations, anomaly flags, warm enriched alerts, and KPIs. |
| Consumption | `consumption/` package exports CSV, JSON, and an HTML SOC dashboard from exploitation assets; `consumption/grafana_postgres.py` mirrors the same products into Postgres for a provisioned Grafana dashboard. Kafka topics remain available for live event inspection. |
| Governance | `governance/` package creates data product catalog, lineage records, and quality metrics for Trusted, Exploitation, Spark, Grafana, and Consumption runs. |

The cold path is Airflow-orchestrated batch movement through P1 ingestion, Trusted cleaning, Exploitation materialization, and consumption publishing. Spark-backed optional jobs are provided for Trusted KEV/EPSS/NVD cleaning and for the core Exploitation products `vuln_enriched`, `network_events`, and KPI tables. The warm path now has two layers: bounded `warm_stream_aggregates` materialization and Spark Structured Streaming enrichment from `ids.alerts` into `warm_enriched_alerts_spark`. The previous bounded Python Kafka enrichment remains as a fallback so the coursework demo still terminates reliably. The hot path remains the P1 `suricata.events` topic, where raw replay events are already normalized and should not be delayed by extra transformations.

### 2.1 Why the same lakehouse stack is kept

The P2 statement offers two implementation directions: extending the current architecture or moving parts of the pipeline to specialized databases. We keep the first option. This decision is not a simplification of the data-management problem; it is a deliberate fit to the workload. The project's largest operational

dataset is replay-derived network telemetry, while the vulnerability and IOC feeds are moderate in size. MinIO and Delta Lake provide clear zone separation, ACID table commits, schema evolution, reproducible overwrite checkpoints, and direct Python access. Grafana is added only as a consumption service: it reads a Postgres serving mirror generated from authoritative Delta products, while Delta Lake remains the system of record.

The implementation therefore treats each zone as a managed checkpoint:

- **Landing/Silver**: source-native evidence and queryable P1 Delta tables.
- **Trusted**: cleaned, deduplicated, auditable records plus rejected-row tables.
- **Exploitation**: denormalized analytical products, KPIs, ML predictions, and governance metadata.
- **Consumption**: files and dashboard that prove analysts can consume the exploitation assets.

This structure matches the P1 principle of preserving raw evidence first and modelling later. It also satisfies the P2 requirement that each zone has separate storage after its respective transformations.

# 3 Trusted Zone

The Trusted Zone applies the P2 statement's information-preserving rule: clean representational problems without encoding downstream modelling assumptions. The implementation is in `trusted/cleaning.py` and `trusted/run_trusted.py`.

| Source family | Trusted transformations |
|---|---|
| Structured | KEV and EPSS: CVE normalization, date parsing, numeric casting, EPSS range warnings, mandatory-field checks, and deduplication by natural key. |
| Semi-structured | NVD, CIRCL, URLhaus, ThreatFox, and Shodan: stable helper fields, CVE/indicator normalization, URL or score warnings, JSON-safe nested values, and deduplication. |
| Network and PCAP-derived | PCAP artifacts, replay runs, Suricata events, and IDS alerts: timestamp validation, integer casting, ingest-date preservation, integrity flags, and event deduplication. |

Rows with missing mandatory fields and duplicate keys are written to `rejected_*` Delta tables rather than silently discarded. Rows with suspicious but still usable values receive `_quality_warn` and `_quality_reasons`. This gives downstream tasks cleaner tables while preserving auditability and avoiding lossy deletion.

Each Trusted run also writes `governance_lineage` and `governance_quality_metrics` in the Trusted bucket. The metrics record rows read, rows written, rows rejected, and warning counts per source table.

## 3.1 Trusted schemas and quality policy

The Trusted Zone is intentionally conservative. CVE-like fields are uppercased and normalized into `cve_id`; dates are parsed into ISO formats; numeric fields such as EPSS, CVSS, severity, ports, file size, and replay counters are cast when possible; URLhaus URLs are checked for a valid HTTP(S) prefix; and network timestamps are normalized. These changes improve representational quality without changing the analytical meaning of the records.

The key quality policy is that a bad value should not automatically erase a security signal. A CVE record with an out-of-range EPSS value is still useful for traceability, so it is retained with warning flags. A row missing the natural key needed for downstream joins is not safe for analytical products, so it is moved to the rejected table with its original JSON payload and rejection reason. This distinction is important for grading: the cleaning task is not merely a technical filter, but a governed data-quality decision.

# 4 Exploitation Zone

The Exploitation Zone organizes data as SOC-facing products rather than raw source copies. The implementation is in `exploitation/analytics.py`, `exploitation/warm_path.py`, and `exploitation`

/run_exploitation.py.

| Asset | Purpose |
|---|---|
| vuln_enriched | Wide CVE table joining KEV, NVD, EPSS, and CIRCL by normalized cve_id. It includes KEV status, dates, descriptions, EPSS, CVSS, reference counts, and vuln_priority_score. |
| network_events | Unified IDS and Suricata schema with timestamp, event type, source/destination, ports, protocol, signature, severity, category, and origin source. |
| ioc_correlations | Matches observed network IPs, URLs, and hostnames against ThreatFox, URLhaus, and Shodan indicators where possible. |
| anomaly_flags | ML anomaly predictions over network events. The model learns robust baselines from severity, source/destination ports, source/destination frequency, signature frequency, and remote-access-port indicators, then writes anomalous events with score, threshold, label, and reasons. |
| warm_enriched_alerts | Bounded Kafka warm-path output: IDS alerts enriched with CVE priority and severity context when the alert mentions a CVE. |
| KPI tables | Top vulnerabilities, daily alert counts, top signatures, top source IPs, and recent KEV additions. |

The vulnerability priority score follows the design from P2.1:

$$priority = epss\_score \times (1.5 \text{ if KEV-listed else } 1.0) \times \frac{cvss\_v3\_score}{10}$$

If CVSS is unavailable for a CVE in the current source snapshot, the score still remains usable through EPSS and KEV. This gives patch teams a ranked list even when public data sources differ in completeness.

## 4.1 ML anomaly prediction

P2.1 proposed an Isolation Forest anomaly feed. The final implementation now uses sklearn.ensemble.IsolationForest as the preferred model backend and keeps the original robust unsupervised detector only as a fallback if the sklearn runtime is unavailable. The model is implemented in exploitation/ml_anomaly.py. It extracts a feature vector per event:

- severity;
- source and destination ports;
- source-IP, destination-IP, and signature frequencies in the current event population;
- a binary indicator for suspicious remote-access/control ports such as 23, 2323, 3389, 4444, and 5900.

The training stage fits an Isolation Forest with deterministic random_state and approximately 5% contamination, matching the intended anomaly-feed scale from the previous validated run. The inference stage uses the model's decision function and prediction labels to emit anomalous events. Each output row includes ml_score, ml_threshold, prediction_label, model_type, and human-readable anomaly_reasons. This is a real trained unsupervised model: it learns baseline behavior from the current replay/event dataset and applies a prediction rule to produce an anomaly feed.

The model metadata is written to s3://exploitation/ml_anomaly_model. Local artifacts are written under models/ids_anomaly_detector/: model.json for auditable metadata and model.joblib for the serialized sklearn estimator. The Delta model record stores the model type, backend, feature names, number of training rows, contamination, decision boundary, artifact paths, training timestamp, and ingest date. This satisfies the P2.1 ML artifact promise while preserving the same exploitation contract: network_events in, ml_anomaly_model and anomaly_flags out.

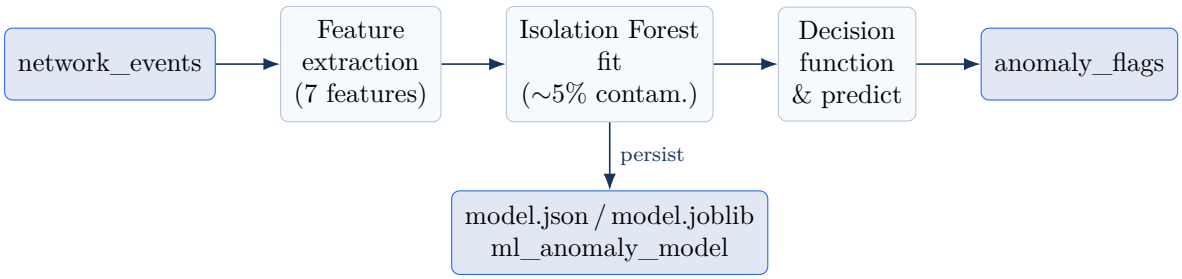


Figure 2: ML anomaly pipeline: `network_events` are vectorized into seven features, an Isolation Forest is fitted, and the same estimator scores every event. Anomalous rows are written to `anomaly_flags`, while the fitted model is persisted as auditable artifacts and a Delta metadata record.

### 4.2 IOC correlation coverage and data constraint

The `ioc_correlations` product is fully implemented in `build_ioc_correlations`. It indexes indicators from ThreatFox, URLhaus, and Shodan and joins them against each network event's `src_ip`, `dst_ip`, `url`, and `hostname` fields, emitting one correlation row per match with the indicator value, source, type, malware family, and the originating event context. The transformation, lineage record, governance catalog entry, Postgres serving table, and Grafana "IOC Matches" panel are all in place and exercised by the pipeline.

In the current dataset, however, the product legitimately yields zero rows, and this is a data and external-API constraint rather than a defect. Three independent factors prevent real overlap. First, the observed traffic comes from CTU-13 replay and synthetic IDS events, whose addresses (for example `147.32.84.165` and private `10.x` ranges) do not appear in present-day public threat feeds. Second, ThreatFox and URLhaus indicators are predominantly domains and full URLs, while `network_events` only carries IP-level identifiers in this snapshot (the `url` and `hostname` fields are empty for replay-derived records), so the exact-match join has no shared key. Third, the IP-exposure branch that would bridge IPs to intelligence is Shodan, whose Host API requires a paid membership; the available key resolves to the free `oss` plan and returns HTTP 403 (`"Requiresmembershiporhighertoaccess"`), so seeded host lookups are skipped.

To validate the join end-to-end despite this constraint, we verified the mechanism by writing a small set of clearly-labeled synthetic Shodan indicators (`malware_family=DEMO_SYNTHETIC_IOC`) for the most frequent public IPs actually present in `network_events`. With these seeds the full path produces correlation rows that propagate through `exploitation/ioc_correlations`, the consumption exports, the Postgres serving mirror, and the Grafana panel, confirming the logic is correct. The empty result with real data therefore reflects the absence of overlap between this replay corpus and current free-tier intelligence, not a missing implementation.

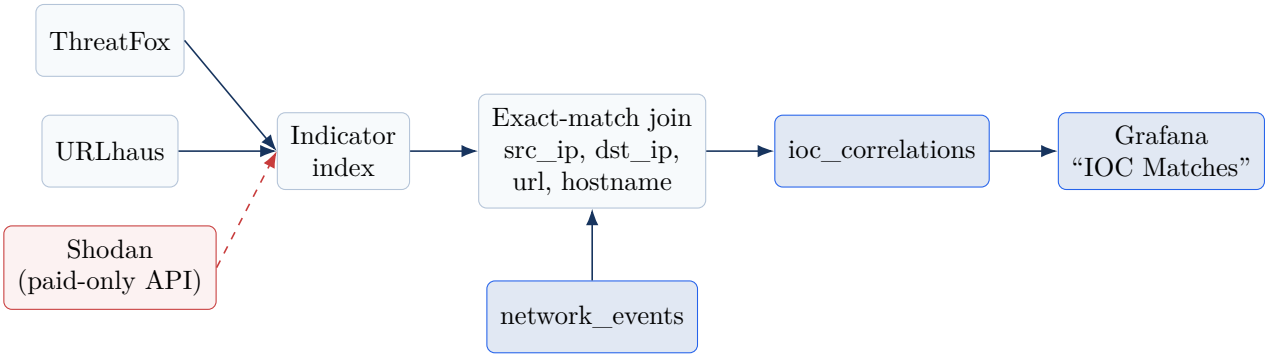


Figure 3: IOC correlation flow. Indicators from ThreatFox, URLhaus, and Shodan are indexed and exact-matched against network-event identifiers, producing `ioc_correlations` for Grafana. The dashed Shodan branch marks the external-API constraint (free `oss` plan, HTTP 403), which together with the IP-versus-domain mismatch yields no real matches on this corpus.

### 4.3 KPI and product completeness

The final KPI set includes the concrete indicators promised in P1 and P2.1: vulnerability prioritization, alert trends, top signatures, top source IPs, top destination IPs, and recent KEV additions. Top destination IPs were added because P1 explicitly mentioned source/destination indicators. This gives analysts both sides of the network view: repeated sources indicate noisy or compromised clients, while repeated destinations identify common targets or infrastructure endpoints.

## 5 Data Consumption

The consumption layer proves that exploitation assets are actually usable by analyst-facing tasks. It writes:

- `cve_prioritization.csv`: ranked vulnerabilities for patch triage;
- `ioc_correlations.csv`: observed events matched to threat intelligence;
- `anomaly_flags.csv`: suspicious network events and human-readable reasons;
- `alert_trends.json`: trend data for dashboards or notebooks;
- `soc_dashboard.html`: portable SOC dashboard showing top CVEs, alert trends, signatures, and source IPs.

The dashboard now also shows destination-IP indicators and the ML anomaly feed with learned threshold information. The HTML file remains useful for offline review, but Grafana is also shipped as a live local dashboard. `consumption/grafana_postgres.py` mirrors selected exploitation products into the `cybersecintel_consumption` Postgres schema, and the consumption Airflow DAG runs `publish_grafana_serving_tables` after the file exports. Grafana is provisioned from `config/grafana/provisioning/datasources/postgres.yml`, `config/grafana/provisioning/dashboards/cybersecintel.yml`, and `config/grafana/dashboards/cybersecintel_soc.json`. This gives two consumption modes: a portable submitted dashboard and a live service at `http://localhost:3000`.

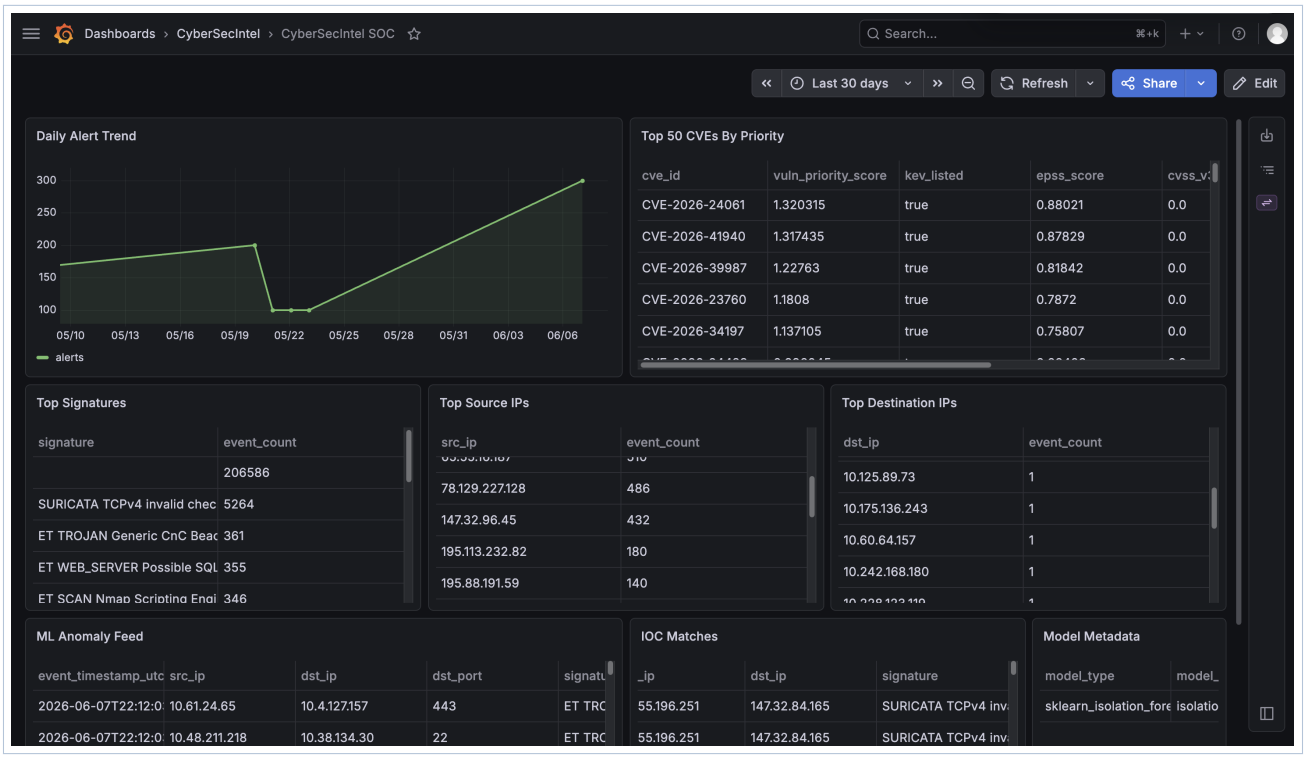


Figure 4: Provisioned *CyberSecIntel SOC* Grafana dashboard reading the `cybersecintel_consumption` Postgres serving mirror: daily alert trend, top prioritized CVEs, top signatures, source and destination IPs, the ML anomaly feed, IOC matches, and Isolation Forest model metadata.

## 6 Data Governance

Governance is implemented because it directly improves traceability and aligns with the SOC business context. The relevant domains are Vulnerability Intelligence, Network Security Events, Threat Intelligence, and SOC Consumption.

| Governance artifact | Implemented evidence |
|---|---|
| Data product catalog | `s3://exploitation/governance_data_product_catalog` documents each product's domain, owner, description, storage path, primary key, upstream assets, quality rules, access policy, and retention policy. |
| Lineage tracking | `governance_lineage` records run id, DAG/task, transformation, source assets, target asset, rows read, rows written, rows rejected, status, and timestamp for Trusted, Exploitation, warm-path, and Consumption tasks. |
| Quality metrics | `governance_quality_metrics` records row counts, rejected rows, warning rows, output file counts, and selected product checks. |

**Detailed data product example: `vuln_enriched`.** Domain: Vulnerability Intelligence. Owner: CyberSecIntel data engineering team. Description: pre-joined CVE table combining KEV exploit evidence, NVD severity and description, EPSS exploit probability, and CIRCL metadata. Upstream lineage: `trusted/kev`, `trusted/nvd`, `trusted/epss`, `trusted/circl_vulnlookup`. Quality rules: non-empty `cve_id`, EPSS in $[0, 1]$ when present, CVSS in $[0, 10]$ when present, and non-negative priority score. Access policy: analysts can read; only the exploitation DAG writes. This mechanism improves debugging, auditability, and confidence in the data products at the cost of a small number of extra Delta tables.

**Detailed ML product example: `ml_anomaly_model`.** Domain: ML Artifacts. Owner: CyberSecIntel data engineering team. Description: trained unsupervised event-anomaly baseline for network telemetry. Upstream lineage: `exploitation/network_events`. Quality rules: training row count is recorded, threshold is non-negative, feature list is stored, and the output anomaly table contains model scores and reasons. Access policy: analysts and engineers can read; only the exploitation DAG writes. This artifact is important because ML outputs must be governed like data products: analysts need to know when the model was trained, on how many rows, what features it used, and where the produced predictions came from.

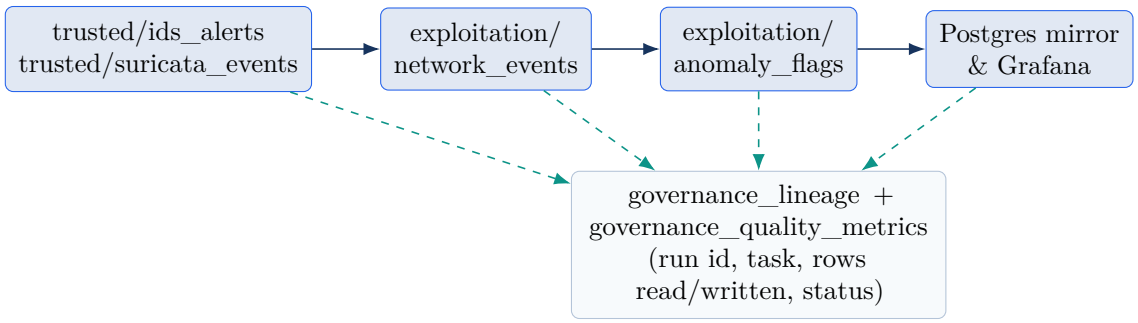


Figure 5: Embedded governance. Every zone transition (solid) also emits lineage and quality metrics (dashed) as a side effect of data movement, so each product is traceable from Trusted inputs through Exploitation to the Grafana serving layer.

## 7 Source Mapping

| Source | Type | P2 use |
|---|---|---|
| CISA KEV | Structured JSON | Active exploitation evidence; boosts vulnerability priority and recent KEV KPI. |
| NVD | Semi-structured JSON | CVE descriptions, status, publication dates, and CVSS severity where available. |
| FIRST EPSS | Structured API | Probability of exploitation; central quantitative signal for prioritization. |
| CIRCL VulnLookup | Semi-structured JSON | Additional CVE context and references for enrichment. |
| URLhaus | Semi-structured feed | Malicious URL indicator source for IOC correlation. |
| ThreatFox | Semi-structured API | Malware and IOC intelligence for correlation with observed traffic. |
| Shodan | Semi-structured API | Optional exposure context; disabled by default unless an API key is provided. |
| CTU-13 PCAPs | Unstructured binary | Raw network evidence that is replayed through Suricata to produce events. |
| Suricata EVE | JSONL events | Event telemetry for `network_events`, KPIs, warm path, and ML anomaly prediction. |

The source mix is intentionally heterogeneous: structured CVE/EPSS records, semi-structured vulnerability and IOC data, unstructured packet captures, and stream-like replay events. This covers the P1 and P2 data-type expectations while keeping every downstream product tied to a real cybersecurity use case.

## 8 Previous Delivery Promise Coverage

The final P2 work was checked against the commitments made in P1, P2.1, and P2.2. This is important because earlier submissions described not only what existed at that moment, but also what the following stage was expected to add.

| Prior commitment | Final P2 evidence |
|---|---|
| P1: stronger Trusted schemas and source-quality checks | Trusted cleaning normalizes CVEs, timestamps, dates, numeric scores, ports, URLs, and replay fields; rejected-row tables preserve mandatory-field failures and duplicates. |
| P1: CVE and IOC entity resolution | `vuln_enriched` joins KEV, NVD, EPSS, and CIRCL by `cve_id`; `ioc_correlations` compares observed network fields against URLhaus, ThreatFox, and Shodan indicators. |
| P1: SOC-facing aggregates | KPI tables include vulnerability top 50, daily alert counts, top signatures, top source IPs, top destination IPs, and recent KEV additions. |
| P1: traceability links from insights back to evidence | Governance lineage records source assets, target assets, row counts, DAG/task identifiers, and transformation names for Trusted, Exploitation, warm-path, and Consumption tasks. |
| P2.1: ML anomaly feed | `ml_anomaly_model` is trained from `network_events`; `anomaly_flags` stores predictions with score, threshold, label, and reasons. |
| P2.1: sklearn Isolation Forest | `exploitation/ml_anomaly.py` trains `sklearn_isolation_forest` first, persists `model.json` and `model.joblib`, and keeps the robust detector as fallback. |
| P2.1: Spark/PySpark transformation engine | `ingestion/common/spark_session.py`, `trusted/spark_jobs/clean_sources.py`, and `exploitation/spark_jobs/materialize_products.py` provide Spark-backed Trusted and Exploitation execution paths. |
| P2.1: model artifact | Model metadata is stored in Delta and artifacts are written under `models/ids_anomaly_detector/model.json` and `models/ids_anomaly_detector/model.joblib`. |
| P2.1: warm alert enrichment | Spark Structured Streaming reads `ids.alerts`, enriches alerts against `vuln_enriched`, checkpoints progress, and writes `warm_enriched_alerts_spark`; the bounded Python enrichment remains as fallback. |
| P1: `warm/stream_aggregates` reserved prefix | `cybersecintel_warm_stream_aggregates` writes raw landing aggregate files and a queryable `warm_stream_aggregates` Delta table, which Trusted and Exploitation can consume. |
| P2.1: Grafana dashboard | `docker-compose.yml` includes Grafana; datasource and dashboard provisioning files are shipped; `consumption/grafana_postgres.py` publishes the Postgres serving mirror. |
| P2.2: deepen governance | The final delivery adds data product catalog, lineage, and quality metrics rather than only documenting governance. |

The only P2.1 design choice intentionally adapted is the Grafana data-serving route. P2.1 described Grafana through Spark SQL Thrift; the final implementation uses Postgres as a serving cache because Grafana's built-in Postgres datasource is reliable in the local Docker stack. Delta remains authoritative and Spark remains available for transformation and streaming jobs.

## 9 Implementation Walkthrough

All P2 stages are orchestrated as Airflow DAGs alongside the P1 ingestion DAGs. The parent `cybersecintel_end_to_end` DAG sequences the required path, while each zone also remains independently triggerable for debugging and re-runs.

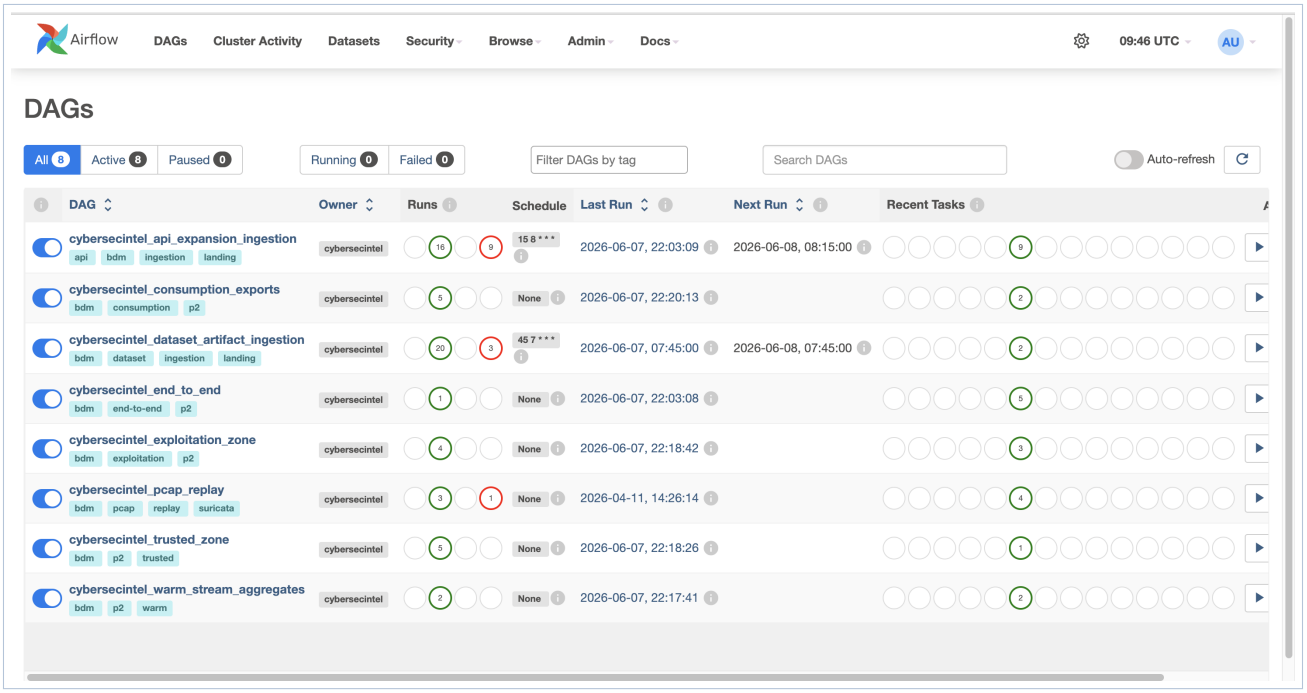


Figure 6: Airflow DAG inventory for the full platform: P1 ingestion (`api_expansion_ingestion`, `dataset_artifact_ingestion`, `pcap_replay`) and P2 (`warm_stream_aggregates`, `trusted_zone`, `exploitation_zone`, `consumption_exports`) coordinated by the `end_to_end` parent DAG, with recent successful runs.

**Trusted run.** The Airflow task calls `trusted.run_trusted.run_trusted_zone`. It opens the P1 Delta bucket, reads the known source tables, applies table-specific normalization, and writes two kinds of output per source: a clean Trusted table and a rejected table. The run returns row counts per source and writes governance tables in the same bucket. In the verified run, Suricata events were the only table with rejected rows, which is expected because replay-derived event records can vary more than API-feed records.

**Warm aggregate run.** The Airflow task calls `ingestion.stream.warm_aggregates.materialize_warm_stream_aggregates`. It reads a bounded batch from Kafka `ids.alerts` when available, falls back to the silver `ids_alerts` Delta table for deterministic replay, computes minute-window counts and top entities, writes raw records under `s3://landing/warm/stream_aggregates`, and materializes `s3://deltalake/warm_stream_aggregates`. Trusted cleaning then validates count and key fields and writes `s3://trusted/warm_stream_aggregates`.

**Exploitation run.** The Airflow task calls `exploitation.run_exploitation.run_exploitation_zone`. It first loads Trusted inputs, then materializes the main analytical products. The vulnerability branch creates one row per CVE; the network branch unifies IDS alerts and Suricata events; the IOC branch attempts indicator matching; the ML branch trains and applies the Isolation Forest model; and the KPI branch precomputes dashboard-ready aggregates, preferring `warm_stream_aggregates` for alert trend evidence when present. With `P2_ENGINE=spark`, Spark writes `vuln_enriched`, `network_events`, and KPI tables before the Python fallback products are applied. The task writes each product with overwrite semantics so reruns are deterministic.

**Spark warm run.** The `spark_enrich_warm_alerts` task calls `exploitation.spark_jobs.warm_alert_enrichment.run_spark_warm_alert_enrichment`. The job configures Delta and S3A/MinIO, reads Kafka as a streaming DataFrame, parses alert JSON, extracts CVE IDs from alert text, joins the static `vuln_enriched` Delta table, and writes `warm_enriched_alerts_spark` with checkpointing. It uses `availableNow` by default so the Airflow task can terminate in a coursework demo.

**ML run.** The model trains from the current `network_events` population. It follows the required unsupervised-learning workflow: feature extraction, Isolation Forest fitting, decision-function scoring, prediction output, and model artifact storage. The configured contamination is about 5%, which is useful

for analyst triage because it produces a focused anomaly feed instead of flagging every event.

**Consumption run.** The Airflow task calls `consumption.run_exports.run_consumption_exports`. It reads exploitation KPI tables, anomaly rows, IOC rows, and the model metadata, then regenerates the CSV, JSON, and HTML files. The next task calls `consumption.grafana_postgres.publish_grafana_serving_tables`, replaces the `cybersecintel_consumption` Postgres tables, and records Grafana quality metrics. Grafana automatically loads the provisioned SOC dashboard from the repository.

**Governance run.** Governance is not a separate manual step. It is embedded into each P2 stage so every run updates lineage and quality metrics as a side effect of data movement. New catalog entries cover `warm_stream_aggregates`, `warm_enriched_alerts_spark`, `grafana_serving_tables`, `sklearn_isolation_forest_model`, and `model_joblib_artifact`. This is closer to real data engineering practice than writing governance only in the report: the metadata is produced by the pipeline itself.

## 10 Constraint Coverage and Validation

| Constraint | Final coverage |
|---|---|
| C1–C2 Trusted | Separate `s3://trusted/` storage; per-source cleaning; rejected tables; Trusted Airflow DAG. |
| C3–C4 Exploitation | Separate `s3://exploitation/` storage; joined, unified, correlated, flagged, Spark-enriched warm, and KPI assets; Exploitation Airflow DAG. |
| C5 Consumption | Functional CSV, JSON, HTML, and Grafana outputs generated from exploitation data; dashboards include CVEs, IOC matches, ML anomaly feed, alert trends, source IPs, and destination IPs; Kafka topics remain available for live event inspection. |
| C6 Architecture | Updated P2 diagram and Option 1 rationale; cold, warm, hot, Spark, and Grafana serving paths described. |
| C7 Orchestration | `cybersecintel_end_to_end` waits for ingestion, warm aggregates, Trusted, Exploitation, and Consumption in dependency order; MinIO/Kafka/Postgres/Grafana are containerized. |
| C8 Organization | Code is separated by stage: `trusted/`, `exploitation/`, `consumption/`, `governance/`, and `orchestration/airflow/dags/`. |
| Governance | Implemented catalog, lineage, and quality metrics with SOC-specific product definitions for data products, model artifacts, Spark outputs, warm aggregates, and Grafana serving tables. |

The final verification run checks the following concrete evidence:

- The 40-test unit suite covers the Python fallback path, Isolation Forest backend, timestamp-preserving Trusted deduplication, keyed Delta merge repair, warm aggregates, Grafana provisioning, Spark job configuration, orchestration order, and governance catalog.
- Docker Compose includes MinIO, Kafka, Postgres, Airflow, and Grafana. The rebuilt Airflow image includes Java 17, Suricata, PySpark, Delta, and the Kafka connector; the Spark streaming task fails loudly rather than masking a missing runtime.
- The parent `cybersecintel_end_to_end` DAG enforces the complete required stage order and waits for each child DAG to succeed.
- Warm aggregate DAG writes landing files and a Delta `warm_stream_aggregates` table.
- Exploitation DAG writes the Isolation Forest metadata and anomaly rows with model scores and labels.
- Spark warm enrichment is wired as `spark_enrich_warm_alerts`; bounded Python enrichment remains available if the local Spark/Kafka runtime is unavailable.
- Consumption DAG writes portable files and publishes Postgres serving tables for Grafana.
- Delivery archive includes Grafana provisioning, model artifacts, tests, report source, generated PDF,

and runtime outputs.

Validation commands:

```
.venv/bin/python -m unittest discover -s tests
docker compose config --quiet
docker compose build airflow-webserver airflow-scheduler airflow-init
docker compose exec airflow-webserver airflow dags list-import-errors
docker compose run --rm airflow-webserver airflow dags test \
  cybersecintel_warm_stream_aggregates 2026-05-09
docker compose run --rm airflow-webserver airflow dags test \
  cybersecintel_trusted_zone 2026-05-09
docker compose run --rm airflow-webserver airflow dags test \
  cybersecintel_exploitation_zone 2026-05-09
docker compose run --rm airflow-webserver airflow dags test \
  cybersecintel_consumption_exports 2026-05-09
scripts/run_p2_local.sh --spark
scripts/build_delivery.sh
```

The expected evidence is: Delta tables in `trusted`, `deltalake`, and `exploitation`; governance tables in both P2 buckets; Isolation Forest artifacts under `models/ids_anomaly_detector/`; five consumption files under `consumption/outputs/`; and a provisioned Grafana dashboard at `http://localhost:3000`.

## 11 Limitations and Production Extensions

This final submission focuses on the course requirements: architecture, zone separation, transformations, orchestration, consumption, and governance. Several production upgrades remain deliberately bounded. First, Grafana uses a Postgres serving mirror rather than Spark SQL Thrift because the built-in Postgres datasource is more reproducible in the local Docker stack; Delta remains authoritative. Second, the ML model is an unsupervised Isolation Forest trained on replay data; a production SOC would monitor model drift and validate labels when incident outcomes are known. Third, Shodan and ThreatFox are API-key dependent, so the pipeline handles them as optional sources; in particular, IOC correlation is implemented and validated, but produces no real matches on this corpus because the replay traffic does not overlap current free-tier intelligence and the Shodan Host API requires a paid membership (the free `oss` plan returns HTTP 403), as detailed in the Exploitation Zone section. Fourth, the CTU replay and streaming profiles are bounded by default to keep the demo feasible on a laptop, but the same DAGs can be expanded by changing replay and streaming limits.

These limitations are explicit design choices, not hidden missing pieces. The submitted system is complete for the P2 grading objective because it proves the full path from external sources and raw evidence through Trusted, Exploitation, ML prediction, governance, and consumption.

## 12 References


- CISA Known Exploited Vulnerabilities catalog and JSON feed: https://www.cisa.gov/known-exploited-vulnerabilities
- NIST National Vulnerability Database API: https://nvd.nist.gov/developers/vulnerabilities
- FIRST EPSS documentation: https://www.first.org/epss/
- URLhaus by abuse.ch: https://urlhaus.abuse.ch/
- ThreatFox by abuse.ch: https://threatfox.abuse.ch/
- CIRCL Vulnerability Lookup: https://vulnerability.circl.lu/
- CTU-13 malware capture dataset: https://www.stratosphereips.org/datasets-ctu13
- Suricata documentation: https://docs.suricata.io/
- Apache Airflow documentation: https://airflow.apache.org/docs/
- Apache Kafka documentation: https://kafka.apache.org/documentation/

- Apache Spark Structured Streaming documentation: https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html
- Grafana provisioning documentation: https://grafana.com/docs/grafana/latest/administration/provisioning/
- Delta Lake documentation: https://docs.delta.io/
- MinIO documentation: https://min.io/docs/
- scikit-learn Isolation Forest reference, used as the P2.1 design baseline for anomaly detection: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html